

Генеративные модели

Лекция 3: Вариационные
автокодировщики

Модели со скрытыми переменными

Предположим, что помимо наблюдаемой переменной $\mathbf{x}$ в нашей задаче есть некоторая скрытая переменная $\mathbf{z}$, которая влияет на $\mathbf{x}$

Мы не можем измерить ее напрямую, но знаем, что она есть

Мы можем записать совместную плотность и разложить ее на условную и априорную:

$$p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{x}|\mathbf{z}) \cdot p_{\theta}(\mathbf{z})$$

$p_{\theta}(\mathbf{z})$ – априорное распределение на скрытых переменных

$p_{\theta}(\mathbf{x}|\mathbf{z})$ – условное распределение наблюдаемых данных

Проинтегрируем по всем $\mathbf{z}$, чтобы найти правдоподобие:

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \int p_{\theta}(\mathbf{x}|\mathbf{z}) \cdot p_{\theta}(\mathbf{z}) d\mathbf{z}$$

Модели со скрытыми переменными

Наша цель – по-прежнему максимизация правдоподобия:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^n \log p_{\boldsymbol{\theta}}(\mathbf{x}_i) \quad \rightarrow \quad \boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^n \log \int p_{\boldsymbol{\theta}}(\mathbf{x}_i | \mathbf{z}_i) \cdot p_{\boldsymbol{\theta}}(\mathbf{z}_i) d\mathbf{z}_i$$

Мы заменили одно сложное распределение $p_{\boldsymbol{\theta}}(\mathbf{x})$ на интеграл от двух более простых:

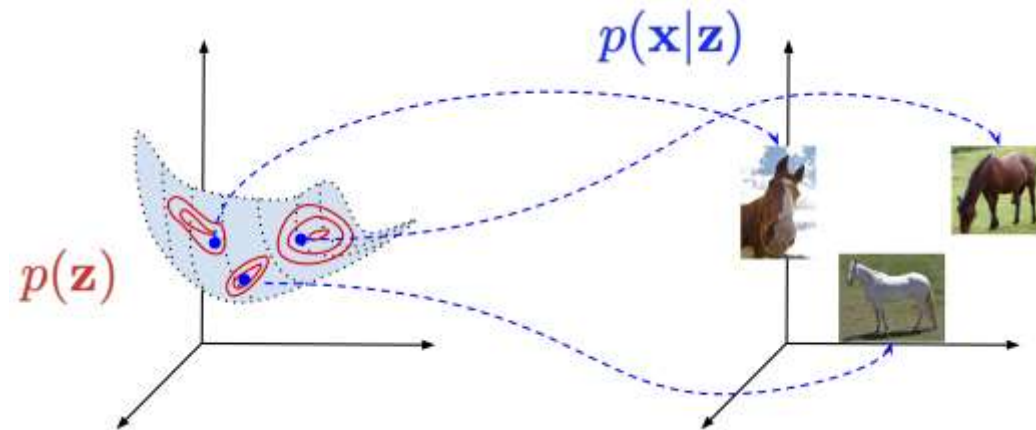
$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \log \int p_{\boldsymbol{\theta}}(\mathbf{x} | \mathbf{z}) \cdot p_{\boldsymbol{\theta}}(\mathbf{z}) d\mathbf{z} \quad \rightarrow \quad \max_{\boldsymbol{\theta}}$$

Выбирая подходящие распределения для $p_{\boldsymbol{\theta}}(\mathbf{x} | \mathbf{z})$ и $p_{\boldsymbol{\theta}}(\mathbf{z})$, мы можем либо получить аналитическое решение для $\boldsymbol{\theta}^*$, либо использовать методы оптимизации

MLE для моделей со скрытыми переменными

Нужно вычислить сложный интеграл:

$$\sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i) = \sum_{i=1}^n \log \int p_{\theta}(\mathbf{x}_i | \mathbf{z}_i) \cdot p_{\theta}(\mathbf{z}_i) d\mathbf{z}_i$$

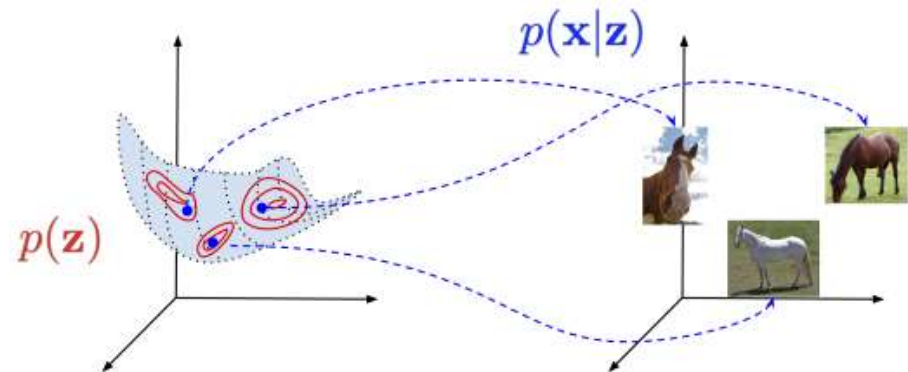


Попробуем оценить этот интеграл методом Монте-Карло:

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x} | \mathbf{z}) \cdot p(\mathbf{z}) d\mathbf{z} = \mathbb{E}_{p(\mathbf{z})}[p_{\theta}(\mathbf{x} | \mathbf{z})] \approx \frac{1}{K} \sum_{k=1}^K p_{\theta}(\mathbf{x} | \mathbf{z}_k)$$

MLE для моделей со скрытыми переменными

- Большинство сэмплов $\mathbf{z}_k$ попадут в «бесполезные» области и декодер сгенерирует шум



Наше априорное распределение $p(\mathbf{z})$ ничего не знает об изображениях $\mathbf{x}$

В многомерном пространстве объем «пустого» пространства огромен по сравнению с объемом полезной области

Чтобы получить надежную оценку интеграла, нам понадобится экспоненциально большое число сэмплов K

Вариационный вывод

Наша цель — оценить $\log p_{\theta}(\mathbf{x})$:

$$\log p_{\theta}(\mathbf{x}) = \log \int p_{\theta}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$

Введем вспомогательное распределение $q(\mathbf{z})$ — произвольную плотность над скрытыми переменными $\mathbf{z}$

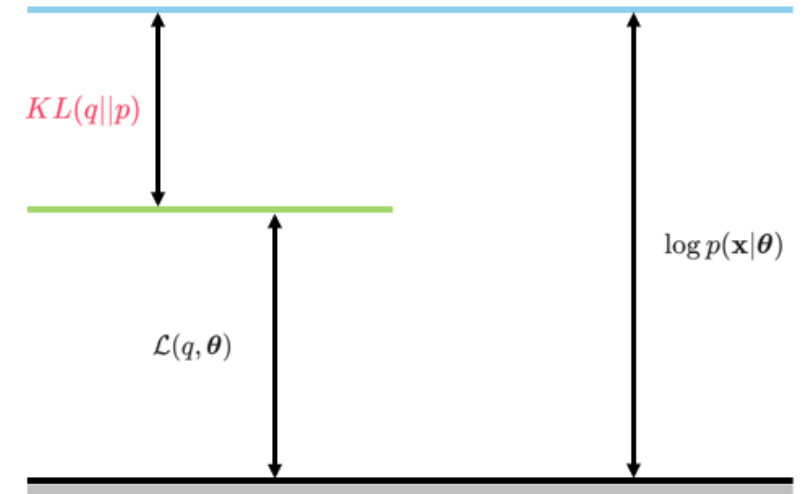
Идея:

Мы не знаем $\log p_{\theta}(\mathbf{x})$, но можем попытаться построить для него некую **нижнюю границу** (*lower bound*), которую будем оптимизировать

Evidence Lower Bound

Зазор между $\log p_{\theta}(\mathbf{x})$ и $\mathcal{L}_{q,\theta}(\mathbf{x})$ равен KL -дивергенции между вариационным распределением $q(\mathbf{z})$ и истинным апостериорным распределением $p_{\theta}(\mathbf{z}|\mathbf{x})$:

$$\log p_{\theta}(\mathbf{x}) - \mathcal{L}_{q,\theta}(\mathbf{x}) = KL(q(\mathbf{z})||p_{\theta}(\mathbf{z}|\mathbf{x}))$$



Выводы

Вместо прямой максимизации правдоподобия $\log p_{\theta}(\mathbf{x})$ будем максимизировать его нижнюю границу $\mathcal{L}_{q,\theta}(\mathbf{x})$ по параметрам вариационного распределения q и параметрам модели θ :

$$\max_{q,\theta} \mathcal{L}_{q,\theta}(\mathbf{x})$$

Максимизируя $\mathcal{L}_{q,\theta}(\mathbf{x})$, мы одновременно стараемся сделать 2 вещи:

1. Увеличить правдоподобие наблюдаемых данных
2. Уменьшить зазор $KL(q(\mathbf{z})||p_{\theta}(\mathbf{z}|\mathbf{x}))$, аппроксимировав $p_{\theta}(\mathbf{z}|\mathbf{x})$ с помощью $q(\mathbf{z})$

План

- Amortized Variational Inference

- Variational Autoencoder

- ELBO Surgery

Amortized Variational Inference

Проблемы вариационного вывода

Цель:

Хотим аппроксимировать сложное апостериорное распределение $p_{\theta}(\mathbf{z}|\mathbf{x})$ с помощью более простого $q(\mathbf{z})$

Проблема 1:

Пространство всех возможных функций $q(\mathbf{z})$ бесконечно

Проблема 2:

Для датасета из N объектов $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, нам нужно решать N отдельных задач оптимизации, чтобы найти $\{q_1(\mathbf{z}_1), \dots, q_n(\mathbf{z}_n)\}$

Амортизированный вариационный вывод

Решение 1, параметризация:

Ограничим поиск $q(\mathbf{z})$ по конкретным семействам распределений:

$$q(\mathbf{z}) \rightarrow q_{\phi}(\mathbf{z})$$

Решение 2, амортизация (Amortized Variational Inference):

Вместо отдельных параметров ϕ для каждого $\mathbf{x}$ обучим единую **нейросеть–кодировщик**, которая по $\mathbf{x}$ будет предсказывать параметры для q :

$$q_{\phi}(\mathbf{z}) \rightarrow q_{\phi}(\mathbf{z}|\mathbf{x})$$

Амортизированный вариационный вывод

Введя эти 2 ограничения мы:

1. Ушли от необходимости считать вариационное распределение для каждого объекта $\mathbf{x}$
2. Введя параметры ϕ , мы признаем, что наше решение может быть не точным и KL -дивергенция между $q_\phi(\mathbf{z}|\mathbf{x})$ и $p_\theta(\mathbf{z}|\mathbf{x})$ может быть больше нуля

Теперь наша задача сводится к нахождению оптимальных ϕ и θ , которые максимизируют $ELBO$

○ Е-шаг:

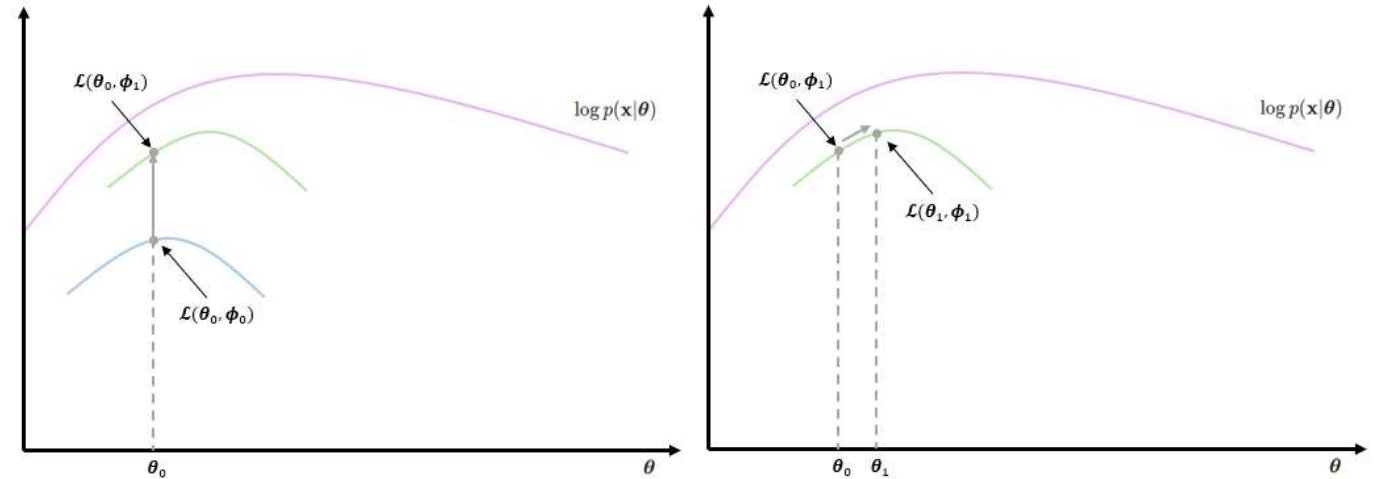
$$\phi_k = \phi_{k-1} + \eta \nabla_{\phi} \mathcal{L}_{\phi, \theta_{k-1}}(\mathbf{x}) \Big|_{\phi = \phi_{k-1}}$$

○ М-шаг:

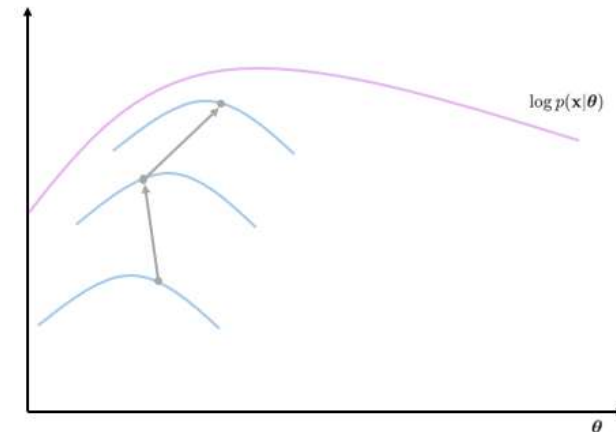
$$\theta_k = \theta_{k-1} + \eta \nabla_{\theta} \mathcal{L}_{\phi_k, \theta}(\mathbf{x}) \Big|_{\theta = \theta_{k-1}}$$

ЕМ алгоритм

- У нас по-прежнему чередование Е- и М-шагов, но теперь с помощью градиентного спуска
- Отличие от обычного ЕМ алгоритма в том, что теперь $KL(q(\mathbf{z})||p_{\theta}(\mathbf{z}|\mathbf{x})) \neq 0$ из-за ограничений кодировщика



- На практике мы будем делать эти шаги **одновременно (совместно)**



ЕМ алгоритм

Как вычислить градиенты $\nabla_{\phi} \mathcal{L}_{\phi, \theta_{k-1}}$ и $\nabla_{\theta} \mathcal{L}_{\phi_k, \theta}$?

$$\phi_k = \phi_{k-1} + \eta \nabla_{\phi} \mathcal{L}_{\phi, \theta_{k-1}}(\mathbf{x}) \Big|_{\phi = \phi_{k-1}}$$

$$\theta_k = \theta_{k-1} + \eta \nabla_{\theta} \mathcal{L}_{\phi_k, \theta}(\mathbf{x}) \Big|_{\theta = \theta_{k-1}}$$

Формула **ELBO** содержит в себе матожидание по вариационному распределению $q_{\phi}(\mathbf{z}|\mathbf{x})$:

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] - KL(q_{\phi}(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$$

Градиент на М-шаге

Считаем градиент по параметрам θ :

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

KL -дивергенция не зависит от θ , так как q параметризуется ϕ , а априорное распределение $p(\mathbf{z})$ обычно фиксировано:

$$\nabla_{\theta} \mathcal{L}_{\phi, \theta}(\mathbf{x}) = \nabla_{\theta} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] = \nabla_{\theta} \int q_{\phi}(\mathbf{z}|\mathbf{x}) \log p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z} = \int q_{\phi}(\mathbf{z}|\mathbf{x}) \nabla_{\theta} \log p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z}$$

Для оценки матожидания будем использовать метод Монте-Карло:

$$\nabla_{\theta} \mathcal{L}_{\phi, \theta}(\mathbf{x}) \approx \nabla_{\theta} \log p_{\theta}(\mathbf{x}|\mathbf{z}^*), \quad \mathbf{z}^* \sim q_{\phi}(\mathbf{z}|\mathbf{x})$$

Поскольку $q_{\phi}(\mathbf{z}|\mathbf{x})$ обусловлено на конкретный $\mathbf{x}$, то его вероятностная масса будет сосредоточена в области пространства $\mathbf{z}$, которая наиболее вероятна для этого $\mathbf{x}$, а значит дисперсия будет сильно ниже, чем при наивном подходе

Градиент на E-шаге

Считаем градиент по параметрам ϕ при фиксированных θ :

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Распределение $q_{\phi}(\mathbf{z}|\mathbf{x})$ зависит от ϕ и мы не можем занести градиент внутрь интеграла:

$$\nabla_{\phi} \mathcal{L}_{\phi,\theta}(\mathbf{x}) = \nabla_{\phi} \int q_{\phi}(\mathbf{z}|\mathbf{x}) \log p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z} - \nabla_{\phi} KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

$$\neq \int q_{\phi}(\mathbf{z}|\mathbf{x}) \nabla_{\phi} \log p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z} - \nabla_{\phi} KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Law of the Unconscious Statistician

- Пусть у нас есть случайная величина $\mathbf{x}$ с распределением $p(\mathbf{x})$
- Создадим новую случайную величину $\mathbf{y} = \mathbf{f}(\mathbf{x})$
- Хотим посчитать матожидание некоторой функции $\mathbf{g}$ от нашей новой переменной $\mathbf{y}$

Сложный путь:

- Находим распределение $p(\mathbf{y})$
- Вычисляем матожидание:

$$\mathbb{E}_{p(\mathbf{y})}[\mathbf{g}(\mathbf{y})] = \int p(\mathbf{y}) \cdot \mathbf{g}(\mathbf{y}) d\mathbf{y}$$

Простой путь (LOTUS trick):

$$\mathbb{E}_{p(\mathbf{y})}[\mathbf{g}(\mathbf{y})] = \mathbb{E}_{p(\mathbf{x})}[\mathbf{g}(\mathbf{f}(\mathbf{x}))] = \int p(\mathbf{x}) \cdot \mathbf{g}(\mathbf{f}(\mathbf{x})) d\mathbf{x}$$

Law of the Unconscious Statistician

- Пусть $x \sim U(0,1)$, $y = x^2$
- Хотим найти $\mathbb{E}[y^2] = \mathbb{E}[x^4]$

Сложный путь:

$$p(y) = p(x) \left| \frac{dx}{dy} \right| = \frac{1}{2\sqrt{y}}, \quad x = f^{-1}(y) = \sqrt{y}, \quad p(x) = 1, \quad \left| \frac{dx}{dy} \right| = \frac{1}{2\sqrt{y}}, \quad y \in (0,1)$$

$$\mathbb{E}_{p(y)}[g(y)] = \int_0^1 p(y)g(y)dy = \int_0^1 \frac{1}{2\sqrt{y}} y^2 dy = \frac{1}{2} \int_0^1 y^{\frac{3}{2}} dy = \frac{1}{5}$$

Простой путь (LOTUS trick):

$$\mathbb{E}_{p(y)}[y^2] = \mathbb{E}_{p(x)}[f(x)^2] = \mathbb{E}_{p(x)}[x^4] = \int_0^1 1 \cdot x^4 dx = \frac{x^5}{5} \Big|_0^1 = \frac{1}{5}$$

Reparameterization trick

Считаем градиент по параметрам ϕ при фиксированных θ :

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Будем использовать **трюк репараметризации (reparameterization trick)** – считать матожидание по распределению, которое не зависит от ϕ

Предположим, что мы можем выразить $\mathbf{z}$ из распределения $q_{\phi}(\mathbf{z}|\mathbf{x})$ как детерминированную функцию $\mathbf{g}$ от некоторой независимой случайной величины ϵ из распределения $p(\epsilon)$:

$$\mathbf{z} = \mathbf{g}_{\phi}(\mathbf{x}, \epsilon) \quad \epsilon \sim p(\epsilon)$$

Пусть $p(\epsilon) = \mathcal{N}(\mathbf{0}, I)$ и функция репараметризации имеет вид:

$$\mathbf{z} = \mathbf{g}_{\phi}(\mathbf{x}, \epsilon) = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \cdot \epsilon$$

$$q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \boldsymbol{\sigma}_{\phi}(\mathbf{x}))$$

Градиент на E-шаге

Считаем градиент по параметрам ϕ при фиксированных θ :

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Используем *reparameterization trick*, чтобы внести оператор градиента внутрь интеграла:

$$\nabla_{\phi} \int q_{\phi}(\mathbf{z}|\mathbf{x}) \log p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z} = \int p(\epsilon) \nabla_{\phi} \log p_{\theta}(\mathbf{x}|\mathbf{g}_{\phi}(\mathbf{x}, \epsilon)) d\epsilon$$

Оценим этот интеграл с помощью метода Монте-Карло:

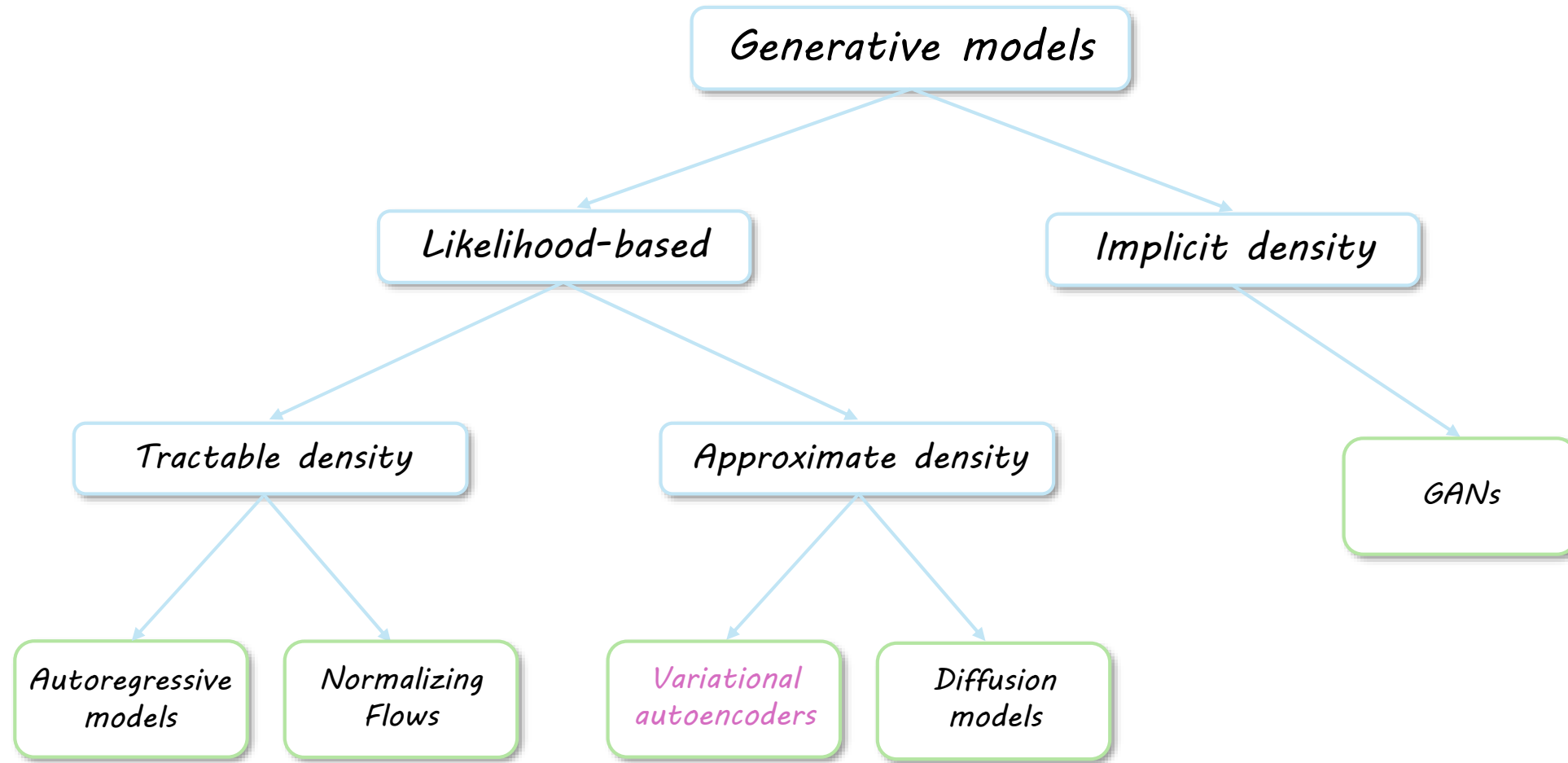
$$\int p(\epsilon) \nabla_{\phi} \log p_{\theta}(\mathbf{x}|\mathbf{g}_{\phi}(\mathbf{x}, \epsilon)) d\epsilon \approx \nabla_{\phi} \log p_{\theta}(\mathbf{x}|\mu_{\phi}(\mathbf{x}) + \sigma_{\phi}(\mathbf{x}) \cdot \epsilon^*), \quad \epsilon^* \sim \mathcal{N}(\mathbf{0}, I)$$

Для KL –дивергенции между двумя нормальными распределениями существует аналитическая формула:

$$\nabla_{\phi} KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) = \nabla_{\phi} KL(\mathcal{N}(\mu_{\phi}(\mathbf{x}), \sigma_{\phi}(\mathbf{x}))||\mathcal{N}(\mathbf{0}, I))$$

Variational Autoencoder

Зоопарк генеративных моделей

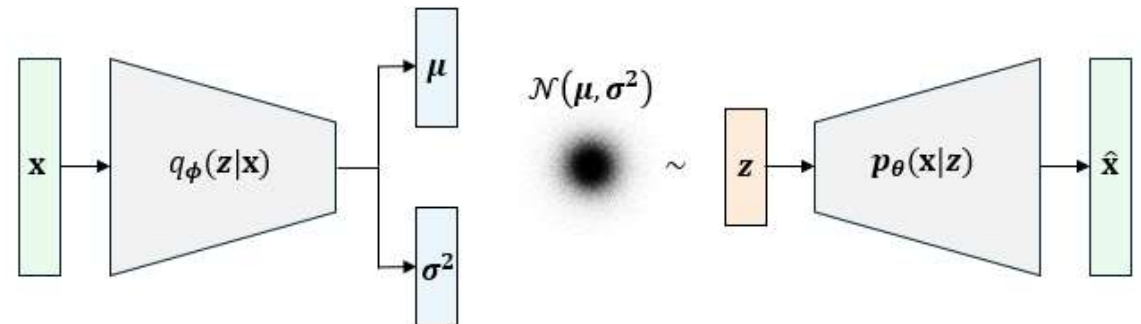


Variational Autoencoder

- *VAE* состоит из двух нейросетей: **кодировщика** и **декодировщика**

Общая идея:

- Одному объекту соответствует не одна точка, а целое распределение $q_{\phi}(\mathbf{z}|\mathbf{x})$
- Кодировщик $q_{\phi}(\mathbf{z}|\mathbf{x})$ выдает не один вектор, а два – вектор средних μ и дисперсий σ^2
- Из этого распределения берется случайная точка $\mathbf{z}$, которая затем подается в декодер для восстановления

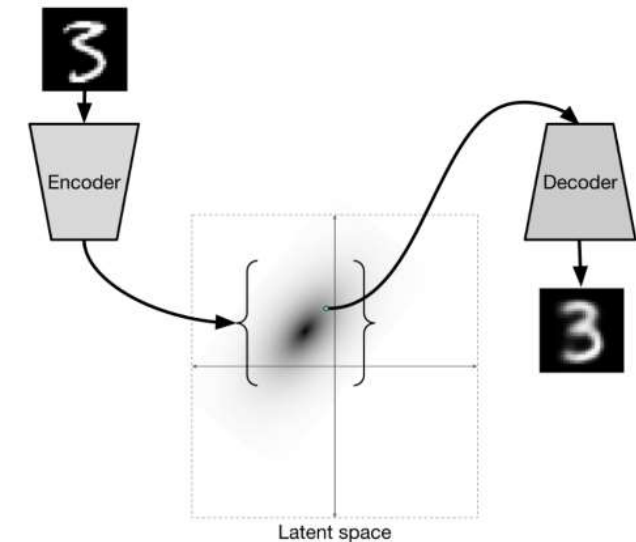


Функция потерь

Обучение *VAE* – поиск компромисса между двумя членами:

$$\text{Loss} = \underbrace{-\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} + \underbrace{KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))}_{\text{Regularization term}}$$

- **Reconstruction term** – насколько хорошо декодер восстанавливает $\mathbf{x}$ из $\mathbf{z}$
- **Regularization term** – насколько распределение $q_{\phi}(\mathbf{z}|\mathbf{x})$ похоже на априорное $p(\mathbf{z})$, заставляет скрытое пространство быть гладким и хорошо организованным



Функция потерь: Reconstruction term

Пусть функция потерь состоит только из *Reconstruction term*:

$$\text{Loss} = -\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})]$$

- У модели одна цель – восстановить $\mathbf{x}$ из $\mathbf{z}$ как можно точнее
- Чтобы помочь декодеру, кодировщик $q_{\phi}(\mathbf{z}|\mathbf{x})$ будет делать $\mathbf{z}$ уникальным для каждого $\mathbf{x}$
- Самый простой способ сделать это – схлопнуть $q_{\phi}(\mathbf{z}|\mathbf{x})$ в одну точку $\rightarrow \sigma^2 = 0$

Итог:

- **Восстановление:** наилучшее
- **Скрытое пространство:** разрушено, поскольку $\mathbf{z}$ будут далеко разбросаны по всему пространству
- **Генерация:** плохая

Функция потерь: Regularization term

Пусть функция потерь состоит только из *Regularization term*:

$$\text{Loss} = KL(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

- У модели одна цель – сделать $q_{\phi}(\mathbf{z}|\mathbf{x})$ неотличимым от $p(\mathbf{z})$
- Самый простой способ сделать это – игнорировать $\mathbf{x}$ и всегда выдавать $\mu = \mathbf{0}$ и $\sigma^2 = \mathbf{1}$

Итог:

- **Восстановление**: плохое, $\mathbf{z}$ не будет содержать никакой информации об $\mathbf{x}$
- **Скрытое пространство**: идеальное, но бесполезное
- **Генерация**: плохая

Обучение VAE

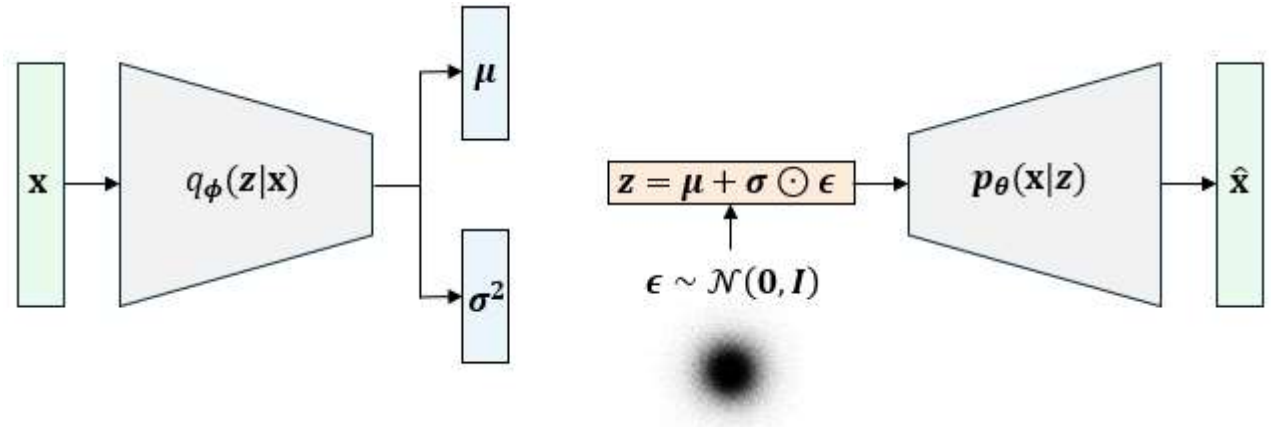
Алгоритм:

- Берем случайный объект $\mathbf{x}$
- Пропускаем его через кодировщик $q_{\phi}(\mathbf{z}|\mathbf{x})$, получаем $\boldsymbol{\mu}_{\phi}(\mathbf{x})$ и $\boldsymbol{\sigma}_{\phi}(\mathbf{x})$
- Вычисляем $\mathbf{z}^*$ с помощью репараметризации:

Берем $\boldsymbol{\epsilon}^* \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

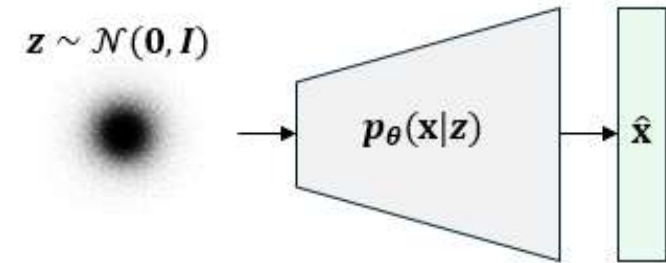
Считаем $\mathbf{z}^* = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \cdot \boldsymbol{\epsilon}^*$

- Пропускаем $\mathbf{z}^*$ через декодер и получаем восстановленный $\hat{\mathbf{x}}$
- Считаем лосс, обновляем параметры ϕ и θ



Генерация VAE

- Кодировщик $q_\phi(\mathbf{z}|\mathbf{x})$ больше не нужен
- Берем случайный вектор $\mathbf{z}^*$ из априорного распределения $\mathcal{N}(\mathbf{0}, \mathbf{I})$
- Подаем $\mathbf{z}^*$ на вход обученному декодеру

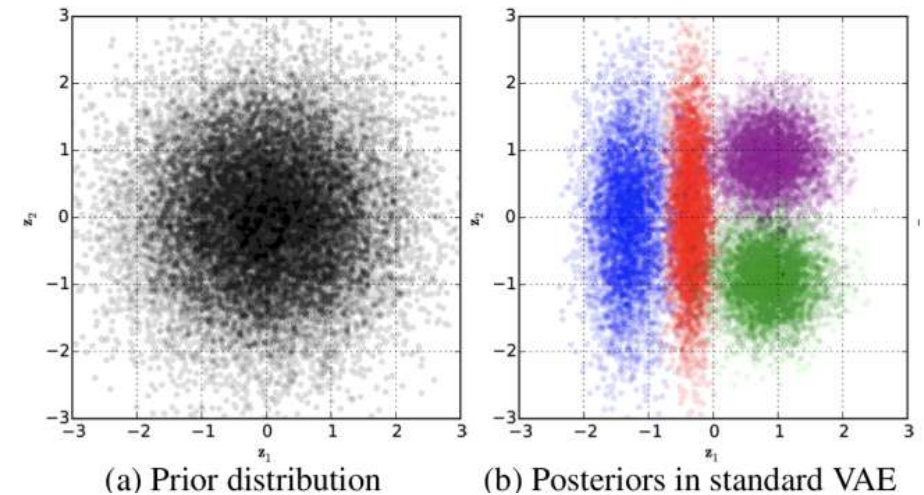


ELBO Surgery

Проблемы VAE

- VAE часто генерирует размытые, усредненные изображения

- Основная причина – несоответствие между априорным распределением $p(\mathbf{z})$ и распределением, которое выучивает кодировщик
- Мы предполагаем, что скрытое пространство должно быть простым $\mathcal{N}(\mathbf{0}, \mathbf{I})$
- Кодировщик на сложных данных будет выучивать сложное, мультимодальное распределение



ELBO Surgery

- Попробуем найти **среднюю KL – дивергенцию** по всей выборке
- Введем среднее вариационное апостериорное распределение $q_{avg}(\mathbf{z}|\boldsymbol{\phi}) = \frac{1}{n} \sum_{i=1}^n q_{\boldsymbol{\phi}}(\mathbf{z}_i|\mathbf{x})$

$$\frac{1}{n} \sum_{i=1}^n KL(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) || p(\mathbf{z})) = \frac{1}{n} \sum_{i=1}^n \int q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z})} d\mathbf{z} =$$

$$\frac{1}{n} \sum_{i=1}^n \int q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \cdot q_{avg}(\mathbf{z}|\boldsymbol{\phi})}{p(\mathbf{z}) \cdot q_{avg}(\mathbf{z}|\boldsymbol{\phi})} d\mathbf{z} = \int \frac{1}{n} \sum_{i=1}^n q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{avg}(\mathbf{z}|\boldsymbol{\phi})}{p(\mathbf{z})} d\mathbf{z} +$$

$$\frac{1}{n} \sum_{i=1}^n \int q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i)}{q_{avg}(\mathbf{z}|\boldsymbol{\phi})} d\mathbf{z} = KL(q_{avg}(\mathbf{z}|\boldsymbol{\phi}) || p(\mathbf{z})) + \frac{1}{n} \sum_{i=1}^n KL(q_{\boldsymbol{\phi}}(\mathbf{z}_i|\mathbf{x}) || q_{avg}(\mathbf{z}|\boldsymbol{\phi}))$$

$$KL(q_{avg}(\mathbf{z}|\boldsymbol{\phi}) || p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}]$$

ELBO Surgery

- **Средняя KL –дивергенция** по всей выборке раскладывается на 2 члена:

$$\frac{1}{n} \sum_{i=1}^n KL(q_{\phi}(\mathbf{z}|\mathbf{x}_i)||p(\mathbf{z})) = KL(q_{avg}(\mathbf{z}|\phi)||p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}]$$

- $q_{avg}(\mathbf{z}|\phi)$ – среднее распределение, которое кодировщик выдает по всей выборке
- $KL(q_{avg}(\mathbf{z}|\phi)||p(\mathbf{z}))$ – маргинальная KL –дивергенция
- $\mathbb{I}_q[\mathbf{x}, \mathbf{z}]$ – взаимная информация (**Mutual Information**)

- **Marginal KL** заставляет $q_{avg}(\mathbf{z}|\phi)$ быть похожим на $p(\mathbf{z})$

- **Mutual Information** измеряет качество кодирования (сколько информации $\mathbf{z}$ содержит об $\mathbf{x}$)

Optimal Prior

ELBO по всей выборке:

$$\frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) = \frac{1}{n} \sum_{i=1}^n \left(\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} [\log p_{\theta}(\mathbf{x}_i|\mathbf{z})] - KL(q_{\phi}(\mathbf{z}|\mathbf{x}_i) || p(\mathbf{z})) \right)$$

$$\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} [\log p_{\theta}(\mathbf{x}_i|\mathbf{z})] - KL(q_{avg}(\mathbf{z}|\boldsymbol{\phi}) || p(\mathbf{z})) - \mathbb{I}_q[\mathbf{x}, \mathbf{z}]$$

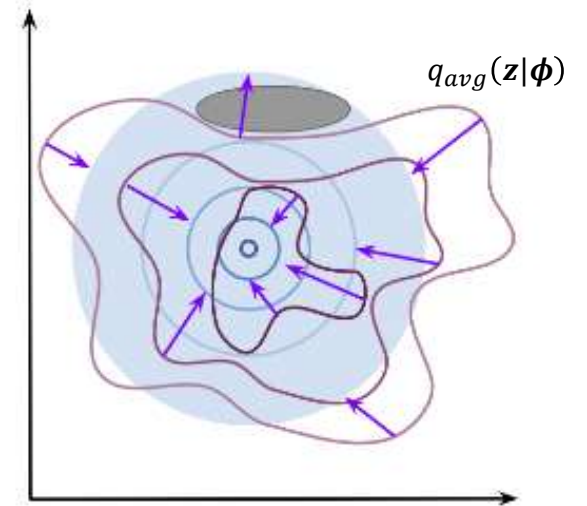
○ $KL(q_{avg}(\mathbf{z}|\boldsymbol{\phi}) || p(\mathbf{z})) = 0$, когда $p(\mathbf{z}) = q_{avg}(\mathbf{z}|\boldsymbol{\phi})$

○ Оптимальное априорное распределение $p(\mathbf{z})$ – это не $\mathcal{N}(\mathbf{0}, I)$, а сложное, мультимодальное среднее апостериорное распределение, которое выучивает кодировщик:

$$p(\mathbf{z}) = q_{avg}(\mathbf{z}|\boldsymbol{\phi}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}_i|\mathbf{x})$$

Проблемы обычного *Prior*

- В *VAE* мы заставляем $q_{avg}(\mathbf{z}|\boldsymbol{\phi})$ соответствовать простому распределению $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$
- Это приводит к **чрезмерной регуляризации** (*over* –



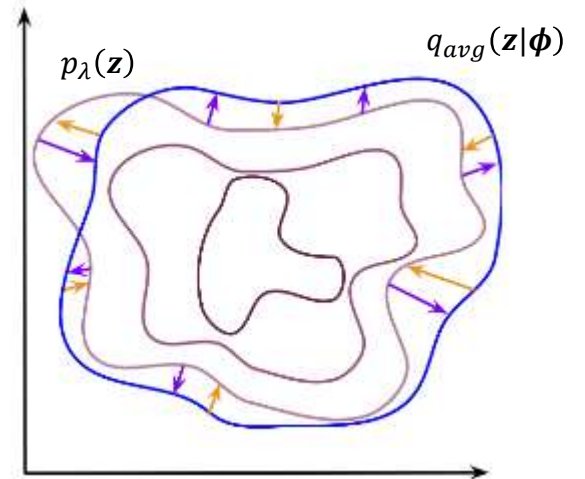
Обучаемый *Prior*

- Оптимальное априорное распределение – это среднее апостериорное распределение:

$$p(\mathbf{z}) = q_{avg}(\mathbf{z}|\boldsymbol{\phi}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}_i|\mathbf{x})$$

- Такой подход ведёт к **переобучению** (*overfitting*) и вычислительно невозможен

- Будем использовать **обучаемый prior** $p_{\lambda}(\mathbf{z})$
- Теперь не только кодировщик подстраивается под *prior*, но и *prior* под кодировщик



Prior на основе нормализующих потоков

Чтобы получить гибкий обучаемый *prior*, можем использовать нормализующие потоки:

$$\log p_{\lambda}(\mathbf{x}) = \log p(\mathbf{z}^*) + \log \left| \det \left(\frac{\partial \mathbf{z}^*}{\partial \mathbf{z}} \right) \right| = \log p(\mathbf{f}_{\lambda}(\mathbf{z})) + \log |\det(\mathbf{J}_f)|$$

В *ELBO* нужно заменить фиксированный *prior* на обучаемый:

$$\begin{aligned} \mathcal{L}_{\phi, \theta, \lambda}(\mathbf{x}) &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z}) + \log p_{\lambda}(\mathbf{x}) - \log q_{\phi}(\mathbf{z}|\mathbf{x})] = \\ &\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z}) + \log p(\mathbf{f}_{\lambda}(\mathbf{z})) + \log |\det(\mathbf{J}_f)| - \log q_{\phi}(\mathbf{z}|\mathbf{x})] \end{aligned}$$

Спасибо за внимание!